

Mock Interview 1

Foundational Round — Core Concepts & Behavioral

Candidate	Viraj Pawar
Experience	3.5 Years Java Full Stack (Capgemini)
Duration	45–55 Minutes
Difficulty	■■ Medium
Stack	Java · Spring Boot · React · MySQL · Docker · K8s

Sections Covered in This Mock Interview

Section 1	HR + Behavioral Round (12 Questions + Model Answers)
Section 2	Core Java + Java 8 Features (13 Questions + Code Snippets)
Section 3	Spring Boot + REST API Design (15 Questions + Scenarios)
Section 4	JPA + Hibernate + SQL (12 Questions + SQL Examples)
Section 5	Microservices + Job Portal Project Deep-Dive (12 Questions)
Section 6	DevOps — Docker / Kubernetes / CI-CD (10 Questions)
Section 7	Coding Round (4 Problems — Easy to Medium)
Section 8	Rapid Fire + Cross Questions (12 Questions)
Section 9	Interviewer Evaluation Guide (Tips to Stand Out)

■ SECTION 1: HR + BEHAVIORAL ROUND

Duration: ~10–12 minutes. This round tests communication, self-awareness, cultural fit, and motivation. Use the STAR method (Situation → Task → Action → Result).

Q1.

Tell me about yourself.

◆ **Model Answer**

My name is Viraj Pawar. I have 3.5 years of experience as a Java Full Stack Developer at Capgemini, where I primarily worked on enterprise-grade applications using Java 17, Spring Boot, Microservices, MySQL, and React. My most notable project was a Job Portal Application built on a microservices architecture — I designed the API Gateway, implemented Role-Based Access Control, and led the containerization using Docker and Kubernetes. I enjoy solving complex backend challenges and I'm now looking to join Accenture to work on larger-scale digital transformation projects. Outside of work, I actively contribute to open-source and stay updated with Java ecosystem trends.

■ *Tip: Keep this under 90 seconds. End with why you want THIS role at THIS company.*

Q2.

Why do you want to join Accenture?

◆ **Model Answer**

Accenture's focus on innovation-led delivery, particularly in cloud-native and microservices transformations, aligns perfectly with my technical journey. I'm especially drawn to Accenture's investment in custom software engineering — working on real business problems at enterprise scale. Additionally, Accenture's culture of continuous learning through certifications, learning boards, and global mobility gives me an environment where I can grow both technically and professionally. I believe my Spring Boot and containerization experience will add value immediately while I scale up on cloud platforms like AWS or Azure.

Q3.

Describe a challenging project and how you handled it.

◆ **Model Answer**

Situation: In my Job Portal project, we faced a critical challenge when our monolithic API started timing out under peak load during campus placement season — 5,000+ concurrent users. Task: I was responsible for diagnosing the bottleneck and proposing a solution within a two-week sprint. Action: I profiled the application using VisualVM and identified that the Resume Service was making synchronous blocking calls to the Notification Service. I introduced asynchronous messaging using Spring's @Async and an event-driven approach, decoupling the services. I also added Redis caching for frequently fetched job listings. Result: Response times dropped from 8 seconds to under 1.2 seconds under the same load, and we deployed the fix without downtime using a rolling update in Kubernetes.

■ *Tip: Always quantify results. Numbers make your story credible.*

Q4.

Tell me about a conflict with a teammate and how you resolved it.

◆ **Model Answer**

In one sprint, a senior developer and I disagreed on the approach to implement pagination for the Job Listings API — he preferred offset-based pagination and I suggested cursor-based pagination for better

performance with large datasets. Rather than escalating, I prepared a small proof-of-concept showing the performance difference at 100K records and shared it in the next tech sync. After reviewing the numbers, the team collectively agreed on cursor-based pagination. The key was bringing data to the discussion instead of just opinions. We also documented the decision in our ADR (Architecture Decision Record) so future team members could understand the reasoning.

Q5.

How do you handle tight deadlines?

◆ **Model Answer**

I break the deliverable into the smallest testable units and prioritize ruthlessly using MoSCoW (Must have, Should have, Could have, Won't have). I communicate proactively with my Scrum Master if any task is blocked so the team can re-plan rather than miss the sprint goal. In one instance, we had a client demo in 48 hours and two critical API endpoints were incomplete. I paired with a colleague, focused only on the happy-path flow needed for the demo, and we delivered on time. I then raised a technical debt ticket to complete edge-case handling in the next sprint.

Q6.

What are your greatest strengths?

◆ **Model Answer**

My top strength is backend architecture thinking — I'm good at breaking down complex requirements into clean, maintainable service boundaries. Secondly, I'm a strong communicator; I can translate technical concepts for non-technical stakeholders. Finally, I'm a self-starter — when I encounter a new technology needed for the project, I don't wait to be trained. For example, when my team adopted Kubernetes, I completed the CKA prep course on my own and became the team's K8s go-to person within a month.

Q7.

What is your biggest weakness?

◆ **Model Answer**

I sometimes over-engineer solutions when a simpler one would suffice. In early projects, I'd default to microservices when a modular monolith would have been faster to deliver. I've been actively working on this by asking myself: 'What is the simplest solution that meets all requirements?' before proposing any design. Exposure to different project sizes at Capgemini has significantly improved this.

■ **Common Mistake: Never say 'I work too hard' — it's clichéd and unbelievable.**

Q8.

Where do you see yourself in 3 years?

◆ **Model Answer**

I see myself growing into a Lead Engineer or Solution Architect role within 3 years. At Accenture, I'd want to start by deeply mastering cloud-native patterns — especially AWS or Azure services — and then take ownership of end-to-end technical delivery for a client engagement. I'm also interested in mentoring junior developers, which is something I've already started doing informally at Capgemini.

Q9.

Tell me about a time you showed leadership without a formal title.

◆ **Model Answer**

During one of our projects, our tech lead went on emergency leave mid-sprint. I volunteered to run the daily standups, maintain the sprint board, and unblock teammates' technical queries. I coordinated with

the client's technical team for clarifications and ensured all critical stories were completed. The sprint velocity actually increased by 15% that week because decisions were being made quickly instead of waiting for approvals. My manager formally recognized this in our next performance review.

Q10.

How do you stay updated with technology?

◆ **Model Answer**

I follow a mix of structured and informal learning. For structured: I've completed Udemy courses on Spring Boot 3, React, and Docker/Kubernetes. For informal: I read Java Weekly by Baeldung, follow the Spring blog, and participate in GitHub discussions on repos I contribute to. I also allocate 30 minutes each morning for reading tech articles. Recently, I've been exploring Spring AI and Project Loom (Virtual Threads) in Java 21.

Q11.

How do you handle feedback or criticism?

◆ **Model Answer**

I genuinely welcome feedback because it's the fastest way to improve. I listen fully without interrupting, ask clarifying questions to understand the specific concern, and then act on it. In a code review, my senior once pointed out that my REST controllers were doing too much business logic. I took that feedback, refactored the service layer accordingly, and even wrote a best-practice guide for our team so we could all align on clean controller design.

Q12.

Tell me about a successful team collaboration.

◆ **Model Answer**

In the Job Portal project, we had a cross-functional team — 3 backend developers, 2 frontend developers, a QA engineer, and a DevOps specialist. I drove the backend design, but I made sure to involve the frontend team early in the API contract definition using OpenAPI/Swagger specs. This eliminated back-and-forth during integration. I also worked closely with the DevOps engineer to define the Kubernetes deployment strategy. The collaboration resulted in a smooth integration phase with zero integration-breaking changes in the last 2 sprints.

■ SECTION 2: CORE JAVA + JAVA 8 FEATURES

Q1.

Explain the four pillars of OOP with real-world examples from your project.

◆ **Model Answer**

Encapsulation: In the User service, I encapsulate user data in the User entity class and expose only necessary methods via getters/setters, keeping internal logic hidden. Inheritance: I have a BaseEntity class with common fields (id, createdAt, updatedAt) that all my JPA entities extend. Polymorphism: I use an interface NotificationService with implementations EmailNotificationService and SMSNotificationService — Spring injects the right one at runtime based on configuration. Abstraction: Repository interfaces hide the JPA implementation details from the service layer.

Q2.

What is the difference between == and .equals() in Java?

◆ **Model Answer**

== compares object references (memory addresses). .equals() compares the content/value if overridden. For String, Integer, and other wrapper classes, always use .equals(). String pool: String s1 = 'abc' and String s2 = 'abc' — here s1 == s2 is true because both point to the same pool literal. But new String('abc') == new String('abc') is false because they're different heap objects.

```
String s1 = "hello"; String s2 = "hello"; String s3 = new String("hello");
System.out.println(s1 == s2); // true (string pool) System.out.println(s1 == s3);
// false (heap object) System.out.println(s1.equals(s3)); // true (content
comparison)
```

Q3.

Difference between ArrayList and LinkedList. When would you use each?

◆ **Model Answer**

ArrayList uses a dynamic array internally — O(1) for random access, O(n) for insertion in the middle. LinkedList uses doubly linked nodes — O(1) for insertion/deletion at head/tail, O(n) for random access. Use ArrayList when you need frequent reads/gets. Use LinkedList when you need frequent insertions/deletions, especially at the beginning. In practice, ArrayList is preferred in 90% of cases due to better cache locality.

Q4.

What are Java 8 Streams? Give a real usage example.

◆ **Model Answer**

Streams provide a declarative, functional-style way to process collections. They support lazy evaluation — intermediate operations are not executed until a terminal operation is called. In my Job Portal, I used streams to filter and transform job listings fetched from the DB.

```
List<Job> activeRemoteJobs = jobs.stream() .filter(j -> j.isActive() &&
j.isRemote()) .sorted(Comparator.comparing(Job::getPostedDate).reversed())
.limit(10) .collect(Collectors.toList()); // With map transformation: List<String>
titles = jobs.stream() .map(Job::getTitle) .distinct()
.collect(Collectors.toList());
```

■ *Tip: Know the difference between map() and flatMap(). flatMap is used when each element maps to a stream (e.g., list of lists).*

Q5.

Explain Optional in Java 8. Why was it introduced?

◆ **Model Answer**

Optional is a container that may or may not hold a non-null value. It was introduced to avoid NullPointerExceptions and to make null-handling explicit in the API contract. Instead of returning null from a method, return Optional.

```
// Repository returns Optional: Optional<User> userOpt =  
userRepository.findByEmail(email); // Safe handling: User user =  
userOpt.orElseThrow(() -> new ResourceNotFoundException("User not found: " +  
email)); // Or with a default: String name =  
userOpt.map(User::getName).orElse("Guest");
```

■ **Common Mistake: Never call .get() without first checking .isPresent(). Use orElseThrow() instead.**

Q6.

What is a Functional Interface and Lambda Expression?

◆ **Model Answer**

A Functional Interface has exactly one abstract method. The @FunctionalInterface annotation enforces this. Lambdas provide a concise implementation of that single abstract method.

```
@FunctionalInterface interface JobFilter { boolean test(Job job); } // Lambda  
usage: JobFilter remoteOnly = job -> job.isRemote(); // Same as method reference:  
JobFilter remoteOnly2 = Job::isRemote; // Built-in: Predicate, Function, Supplier,  
Consumer Predicate<String> isEmpty = s -> !s.isEmpty(); Function<String,  
Integer> strToInt = Integer::parseInt;
```

Q7.

Explain HashMap internals. What happens when two keys have the same hashCode?

◆ **Model Answer**

HashMap uses an array of buckets. When you put a key-value pair, it computes key.hashCode(), applies a secondary hash, and finds the bucket index. If two keys map to the same bucket (hash collision), the entries are stored as a linked list in that bucket. From Java 8+, when the linked list in a bucket exceeds 8 entries, it's converted to a balanced Red-Black tree (TreeNode) for O(log n) instead of O(n) lookup. The default initial capacity is 16 with a load factor of 0.75 — when 75% full, it doubles and rehashes.

Q8.

What is the difference between Comparable and Comparator?

```
// Comparable: natural ordering, implemented IN the class public class Job  
implements Comparable<Job> { @Override public int compareTo(Job other) { return  
this.salary.compareTo(other.salary); } } // Comparator: custom ordering, defined  
OUTSIDE the class Comparator<Job> byTitle = Comparator.comparing(Job::getTitle);  
Comparator<Job> bySalaryDesc = Comparator.comparing(Job::getSalary).reversed();  
jobs.sort(bySalaryDesc.thenComparing(Job::getTitle));
```

Q9.

Explain multithreading — Thread vs Runnable vs ExecutorService.

◆ Model Answer

Thread: Extend Thread class and override run(). Not recommended because Java is single inheritance.
Runnable: Implement Runnable and pass to a Thread — better separation of concern. ExecutorService: Preferred in production. Manages thread pool, lifecycle, and task submission.

```
ExecutorService executor = Executors.newFixedThreadPool(5); Future<String> future = executor.submit(() -> { return fetchJobListings(); // runs in thread pool }); String result = future.get(); // blocks until result is ready executor.shutdown();
```

Q10.

What is the volatile keyword in Java?

◆ Model Answer

volatile ensures that reads/writes to a variable go directly to main memory and are visible across all threads. It prevents thread-local caching of the variable. However, it does NOT make operations atomic. Use AtomicInteger or synchronized blocks for compound operations like count++.

Q11.

Difference between checked and unchecked exceptions?

◆ Model Answer

Checked exceptions extend Exception (not RuntimeException). The compiler forces you to handle them via try-catch or declare them with 'throws'. Example: IOException, SQLException. Unchecked exceptions extend RuntimeException. They're not required to be caught. Example: NullPointerException, IllegalArgumentException. In Spring Boot, prefer custom unchecked exceptions (extend RuntimeException) wrapped in a global @ControllerAdvice.

Q12.

What are method references in Java 8?

```
// 4 types of method references: // 1. Static method reference Function<String, Integer> parse = Integer::parseInt; // 2. Instance method on arbitrary instance Function<String, String> upper = String::toUpperCase; // 3. Instance method on particular instance String prefix = "Job:"; Function<String, String> addPrefix = prefix::concat; // 4. Constructor reference Supplier<ArrayList<Job>> listSupplier = ArrayList::new;
```

Q13.

What is the difference between String, StringBuilder, and StringBuffer?

◆ Model Answer

String is immutable — each concatenation creates a new object. StringBuilder is mutable and not thread-safe — use inside a single thread for better performance. StringBuffer is mutable and thread-safe (synchronized) but slower. In a loop doing 10,000 concatenations, StringBuilder is ~200x faster than String concatenation with +.

■ SECTION 3: SPRING BOOT + REST API DESIGN

Q1.

How does Spring Boot auto-configuration work?

◆ **Model Answer**

Spring Boot scans the classpath for `@EnableAutoConfiguration` (triggered by `@SpringBootApplication`). It reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` and conditionally creates beans using `@ConditionalOnClass`, `@ConditionalOnMissingBean`, etc. For example, if `spring-boot-starter-data-jpa` is on the classpath and you haven't defined a `DataSource` bean, Spring Boot auto-configures one using properties from `application.properties`.

Q2.

Difference between `@Component`, `@Service`, `@Repository`, `@Controller`.

◆ **Model Answer**

All four are specializations of `@Component` — they all register a bean in the Spring context. The differences are semantic and AOP-related. `@Repository` adds exception translation — JDBC/JPA exceptions are wrapped into Spring's `DataAccessException` hierarchy. `@Service` is a marker for business logic. `@Controller` marks a web layer bean and works with `@RequestMapping` to handle HTTP requests. `@RestController` = `@Controller` + `@ResponseBody`.

Q3.

Explain `@Transactional`. What is propagation and isolation?

◆ **Model Answer**

`@Transactional` wraps a method in a database transaction. If the method throws a `RuntimeException`, the transaction rolls back. Propagation defines how transactions interact when methods call each other — `REQUIRED` (default: join or create), `REQUIRES_NEW` (always new transaction), `NESTED` (savepoint). Isolation defines visibility of changes — `READ_COMMITTED` (default in most DBs), `SERIALIZABLE` (highest isolation, prevents phantom reads but slowest).

```
@Transactional(propagation = Propagation.REQUIRED, isolation =
Isolation.READ_COMMITTED, rollbackFor = CustomException.class) public void
applyForJob(Long jobId, Long userId) { // DB operations here – all in one
transaction }
```

Q4.

How do you implement global exception handling in Spring Boot?

```
@RestControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(ResourceNotFoundException.class) public
ResponseEntity<ErrorResponse> handleNotFound( ResourceNotFoundException ex,
HttpServletRequest req) { ErrorResponse err = new ErrorResponse(
HttpStatus.NOT_FOUND.value(), ex.getMessage(), req.getRequestURI() ); return
ResponseEntity.status(HttpStatus.NOT_FOUND).body(err); }
@ExceptionHandler(MethodArgumentNotValidException.class) public
ResponseEntity<ErrorResponse> handleValidation( MethodArgumentNotValidException
ex) { List<String> errors = ex.getBindingResult().getFieldErrors().stream()
.map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
.collect(Collectors.toList()); return ResponseEntity.badRequest().body(new
ErrorResponse(400, errors)); } }
```


Q5.

How do you secure a REST API using Spring Security + JWT?

◆ Model Answer

The flow: Client sends credentials → AuthController validates → generates JWT (signed with secret key) → returns to client. On subsequent requests, client sends JWT in Authorization header. A JwtAuthFilter (extends OncePerRequestFilter) extracts and validates the token, sets SecurityContext. @PreAuthorize('hasRole(ADMIN)') on endpoints enforces RBAC.

■ Tip: In interviews, explain the JWT structure: Header.Payload.Signature and why it's stateless (server doesn't need to store session).

Q6.

Explain @Bean vs @Component.

◆ Model Answer

@Component is a class-level annotation — Spring discovers it via classpath scanning. @Bean is a method-level annotation inside a @Configuration class — you explicitly create and configure the bean. Use @Bean when: you need to configure third-party library classes that you can't annotate, or when you need complex initialization logic.

Q7.

What are Spring profiles? How did you use them?

◆ Model Answer

Profiles allow different configurations for different environments. In my Job Portal, I had application-dev.properties (H2 in-memory DB, detailed logging), application-staging.properties (real MySQL, reduced logging), and application-prod.properties (optimized DB pool, minimal logging). Activated via: spring.profiles.active=prod in environment variable or JVM arg.

```
@Configuration @Profile("dev") public class DevDataSourceConfig { @Bean public
DataSource dataSource() { return new EmbeddedDatabaseBuilder()
.setType(EmbeddedDatabaseType.H2).build(); } }
```

Q8.

How do you validate request bodies in Spring Boot?

```
public class JobApplicationRequest { @NotBlank(message = "Resume URL is required")
@URL(message = "Must be a valid URL") private String resumeUrl; @NotNull @Min(0)
private Integer yearsOfExperience; } // Controller: @PostMapping("/apply") public
ResponseEntity<?> apply(@Valid @RequestBody JobApplicationRequest req) { // if
validation fails, MethodArgumentNotValidException is thrown }
```

Q9.

Difference between @RequestParam and @PathVariable?

◆ Model Answer

@PathVariable extracts values from the URI path: GET /jobs/42 → @PathVariable Long id. @RequestParam extracts from query string: GET /jobs?status=active → @RequestParam String status. @RequestBody deserializes the HTTP body to an object (used with POST/PUT).

Q10.

How do you handle pagination in Spring Boot?

```
// Controller: @GetMapping("/jobs") public Page<JobDTO> getJobs(Pageable pageable)
{ return jobService.findAll(pageable); } // Repository: public interface
JobRepository extends JpaRepository<Job, Long> { Page<Job> findByStatus(JobStatus
status, Pageable pageable); } // Request: GET
/jobs?page=0&size=20&sort=postedDate,desc
```

Q11.

What is Actuator and how did you use it?

◆ Model Answer

Spring Boot Actuator exposes production-ready endpoints for monitoring: /actuator/health (health checks), /actuator/metrics (JVM, HTTP, DB metrics), /actuator/info (app info), /actuator/env (environment properties). In my project, I integrated Actuator with Prometheus (micrometer-registry-prometheus) and visualized metrics in Grafana. I also customized /health to include DB and Kafka connectivity status.

Q12.

Scenario: Your REST API returns 500 errors intermittently under load. How do you debug?

◆ Model Answer

1. Check logs in Kibana/Splunk for stack traces at the 500 error timestamp. 2. Look at Actuator /actuator/metrics for thread pool saturation, DB connection pool exhaustion. 3. Enable SQL logging to spot long-running queries. 4. Check if errors correlate with GC pauses using JVM metrics. 5. Use distributed tracing (Zipkin/Sleuth) to find the slow service. In one actual incident, the root cause was HikariCP connection pool timeout — fixed by increasing pool size and enabling connection timeout logging.

Q13.

How does @Async work in Spring Boot?

```
@EnableAsync // on @SpringBootApplication class @Service public class
NotificationService { @Async("taskExecutor") // runs in separate thread pool
public CompletableFuture<Void> sendWelcomeEmail(String email) { // long running
email logic return CompletableFuture.completedFuture(null); } } // Thread pool
config: @Bean("taskExecutor") public Executor taskExecutor() {
ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
executor.setCorePoolSize(5); executor.setMaxPoolSize(20);
executor.setQueueCapacity(100); return executor; }
```

Q14.

What is the difference between @RestController and @Controller?

◆ Model Answer

@Controller returns a view name (for MVC with Thymeleaf/JSP). @RestController = @Controller + @ResponseBody, meaning every method returns data directly serialized to JSON/XML in the response body. For REST APIs, always use @RestController.

Q15.

How do you configure CORS in Spring Boot?

```
@Configuration public class CorsConfig { @Bean public WebMvcConfigurer
corsConfigurer() { return new WebMvcConfigurer() { @Override public void
addCorsMappings(CorsRegistry registry) { registry.addMapping("/api/**")
```

```
.allowedOrigins("https://jobportal.com")  
.allowedMethods("GET","POST","PUT","DELETE") .allowCredentials(true); } }; }
```

■ SECTION 4: JPA + HIBERNATE + SQL

Q1.

Explain Hibernate's first-level and second-level cache.

◆ **Model Answer**

First-level cache is the Session/EntityManager cache — it's enabled by default. Within a transaction, if you load the same entity twice, Hibernate returns the cached object without another DB hit. Second-level cache is shared across sessions (e.g., Ehcache, Redis). It's optional and configured with `@Cache` annotation. Use it for read-mostly entities that don't change frequently (e.g., lookup tables, categories).

Q2.

What is the N+1 query problem and how do you solve it?

◆ **Model Answer**

N+1 occurs when you load N parent entities and then execute N additional queries to fetch each child collection. Example: Loading 100 jobs and then fetching applicants for each = 101 queries. Solutions: (1) JOIN FETCH in JPQL — loads all in one query. (2) `@EntityGraph` — declarative eager loading for specific use cases. (3) Batch fetching with `@BatchSize`.

```
// N+1 Problem: List<Job> jobs = jobRepo.findAll(); // 1 query jobs.forEach(j ->
j.getApplicants().size()); // N queries! // Fix with JOIN FETCH: @Query("SELECT j
FROM Job j JOIN FETCH j.applicants WHERE j.isActive = true") List<Job>
findActiveJobsWithApplicants(); // Fix with @EntityGraph:
@EntityGraph(attributePaths = {"applicants", "skills"}) List<Job>
findByStatus(JobStatus status);
```

Q3.

Difference between Lazy and Eager loading. Which should you prefer?

◆ **Model Answer**

LAZY: The association is loaded only when accessed. Efficient — avoids unnecessary DB calls. Risk: `LazyInitializationException` if accessed outside a session. EAGER: Association is always loaded with the parent. Can cause performance issues if the collection is large. Best practice: Default to LAZY for all `@OneToMany` and `@ManyToMany`. Use EAGER sparingly, only for `@ManyToOne` or `@OneToOne` where the associated object is always needed.

Q4.

Write a SQL query to find the top 3 most applied-to jobs.

```
SELECT j.title, COUNT(a.id) AS application_count FROM jobs j INNER JOIN
applications a ON a.job_id = j.id WHERE j.is_active = TRUE GROUP BY j.id, j.title
ORDER BY application_count DESC LIMIT 3;
```

◆ **Model Answer**

GROUP BY aggregates applications per job. ORDER BY + LIMIT gives the top 3. In a real scenario, add a date filter (WHERE a.applied_date > DATE_SUB(NOW(), INTERVAL 30 DAY)) to get current month stats.

Q5.

Difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN.

```
-- INNER JOIN: Only matching rows from both tables SELECT u.name, a.applied_date
FROM users u INNER JOIN applications a ON u.id = a.user_id; -- LEFT JOIN: All
users, even those with no applications SELECT u.name, a.applied_date FROM users u
LEFT JOIN applications a ON u.id = a.user_id; -- FULL OUTER JOIN (MySQL: emulate
with UNION) SELECT u.name, a.applied_date FROM users u LEFT JOIN applications a ON
u.id = a.user_id UNION SELECT u.name, a.applied_date FROM users u RIGHT JOIN
applications a ON u.id = a.user_id;
```

Q6.

What is database indexing? How did you use indexes in your project?

♦ **Model Answer**

An index is a data structure (usually B-tree) that speeds up row retrieval. Without index: full table scan $O(n)$. With index: $O(\log n)$. In my Job Portal, I added composite indexes on (status, posted_date) for the jobs table because the most common query was 'get active jobs ordered by date'. I also indexed email in users table for login lookups. Caution: Indexes slow down INSERT/UPDATE/DELETE because the index must be updated too.

Q7.

Explain @OneToMany and @ManyToMany mappings with cascading.

```
@Entity public class Job { @Id @GeneratedValue private Long id;
@OneToMany(mappedBy = "job", cascade = CascadeType.ALL, orphanRemoval = true,
fetch = FetchType.LAZY) private List<Application> applications; @ManyToMany
@JoinTable(name = "job_skills", joinColumns = @JoinColumn(name = "job_id"),
inverseJoinColumns = @JoinColumn(name = "skill_id")) private Set<Skill>
requiredSkills; }
```

Q8.

What is @Transactional readOnly = true and when should you use it?

♦ **Model Answer**

@Transactional(readOnly = true) tells Hibernate and the DB driver that this transaction will only read data. Benefits: Hibernate skips dirty checking (no need to track entity changes for flush), the DB may use read replicas, and connection pools can optimize. Always use it on service methods that only query data.

Q9.

Write a SQL query to find users who have NOT applied to any job.

```
-- Using NOT EXISTS (most efficient): SELECT u.id, u.name, u.email FROM users u
WHERE NOT EXISTS ( SELECT 1 FROM applications a WHERE a.user_id = u.id ); --
Alternative using LEFT JOIN: SELECT u.id, u.name FROM users u LEFT JOIN
applications a ON u.id = a.user_id WHERE a.id IS NULL;
```

Q10.

What are Spring Data JPA derived query methods?

```
public interface JobRepository extends JpaRepository<Job, Long> { // Spring
generates query from method name: List<Job> findByStatusAndIsRemote(JobStatus
status, boolean remote); Optional<Job> findByTitleContainingIgnoreCase(String
keyword); long countByCompanyIdAndStatus(Long companyId, JobStatus status);
List<Job> findTop5ByOrderByPostedDateDesc(); @Query("SELECT j FROM Job j WHERE
j.salary BETWEEN :min AND :max") List<Job> findBySalaryRange(@Param("min") Long
```

```
min, @Param("max") Long max); }
```

■ SECTION 5: MICROSERVICES + JOB PORTAL ARCHITECTURE

Q1.

Walk me through your Job Portal microservices architecture.

◆ **Model Answer**

The system comprised 6 microservices: (1) API Gateway — Spring Cloud Gateway, routes all external traffic, handles JWT validation, rate limiting. (2) User Service — registration, login, profile management. (3) Job Service — CRUD for job postings, search, filtering. (4) Application Service — manages job applications, tracks status (applied/reviewed/rejected). (5) Notification Service — sends email/SMS on application events (event-driven via async calls). (6) Resume Service — handles resume upload/parsing. Each service had its own MySQL schema (database-per-service pattern). Services communicated synchronously via REST (Feign Client) for user lookups and asynchronously via Spring Events/messaging for notifications.

■ *Tip: Draw a mental diagram before answering this. Interviewers love it when you speak fluently about your own architecture.*

Q2.

How did you implement the API Gateway? What does it do?

◆ **Model Answer**

Used Spring Cloud Gateway (reactive, non-blocking). Responsibilities: (1) Routing — routes `/api/jobs/**` to Job Service, `/api/users/**` to User Service. (2) JWT Authentication Filter — validates token before forwarding request. (3) Rate Limiting — using Redis Token Bucket. (4) Circuit Breaker — Resilience4j integration to return fallback response if a downstream service is down. (5) Request logging and correlation ID injection.

Q3.

How did you handle RBAC (Role-Based Access Control)?

◆ **Model Answer**

Three roles: ADMIN, EMPLOYER, JOBSEEKER. JWT token contains the user's role in the claims. The API Gateway validates the token and forwards it. Individual services use `@PreAuthorize('hasRole(EMPLOYER)')` to restrict endpoints. For example, only EMPLOYER can create/edit jobs, only ADMIN can delete users. The Role is stored in the users table and embedded in the JWT at login time.

Q4.

What is a Circuit Breaker? How did you implement it?

◆ **Model Answer**

A Circuit Breaker prevents cascading failures. States: CLOSED (normal), OPEN (failing — short-circuits requests without calling the service), HALF-OPEN (testing if the service has recovered). I used Resilience4j with Spring Boot. If the Job Service failed 5 times in 10 seconds, the circuit opened and the fallback returned cached results.

```
@CircuitBreaker(name = "jobService", fallbackMethod = "fallbackJobs") public
List<JobDTO> getAllJobs() { return jobFeignClient.getAllJobs(); } public
List<JobDTO> fallbackJobs(Exception ex) { log.warn("Job service unavailable, using
```

```
fallback"); return Collections.emptyList(); // or cached data }
```

Q5.

How did you handle distributed data consistency? Did you use SAGA pattern?

◆ **Model Answer**

We used the Choreography SAGA pattern for the job application flow: (1) Application Service saves application with status PENDING. (2) It publishes an ApplicationCreatedEvent. (3) Notification Service listens and sends email. (4) If email fails, Notification Service publishes EmailFailedEvent, Application Service marks notification as failed. Each service is responsible for its own compensating action. We avoided distributed transactions (2PC) because they introduce tight coupling and availability risks.

Q6.

What challenges did you face in the microservices architecture and how did you solve them?

◆ **Model Answer**

Challenge 1: Service discovery — solved using Spring Cloud Eureka (services register themselves, Feign Client resolves URLs). Challenge 2: Configuration management — used Spring Cloud Config Server with a Git repo to centralize all service properties. Challenge 3: Distributed tracing — added Micrometer + Zipkin. Every request got a correlation trace ID, making it easy to trace a request across 4 services in Zipkin's UI. Challenge 4: Database migration management — used Flyway for versioned SQL migrations across all services.

Q7.

How would you scale the Job Service if it received 10x the usual traffic?

◆ **Model Answer**

Horizontal scaling: Kubernetes HPA (Horizontal Pod Autoscaler) adds more replicas when CPU > 70%. Database: Read replicas for SELECT queries, connection pool tuning. Caching: Add Redis cache for frequently accessed job listings (TTL: 5 minutes). Async: Move non-critical processing (analytics, logging) to async event queues. CDN: Serve static assets and non-personalized job listings from CDN. Database: If MySQL becomes a bottleneck, consider sharding by company_id or introducing Elasticsearch for full-text search on job titles/descriptions.

Q8.

How did you deploy using Docker and Kubernetes?

◆ **Model Answer**

Each microservice had a Dockerfile. I wrote a docker-compose.yml for local development with all services, MySQL, and Redis. For Kubernetes: each service had a Deployment (replica: 2), Service (ClusterIP), and ConfigMap/Secret. The API Gateway was exposed via a LoadBalancer Service. I wrote Helm charts for templated deployments. Used kubectl rolling updates for zero-downtime deployments.

Q9.

How do you make microservices communicate? REST vs Messaging — when to use what?

◆ **Model Answer**

REST/Feign: Use when you need an immediate response and strong consistency. Example: Job Service calling User Service to validate if user exists during application submission. Messaging (async): Use when the caller doesn't need an immediate response and loose coupling is preferred. Example: After

application submission, notify the employer via email — Notification Service can process this asynchronously. Rule of thumb: If you need the result to proceed, use REST. If it's a side effect, use messaging.

Q10.

How would you improve your Job Portal architecture if you were to build it again?

◆ Model Answer

1. Add Elasticsearch for job search instead of MySQL LIKE queries — much better performance and relevance scoring. 2. Use an event broker (Kafka or RabbitMQ) instead of synchronous events for better decoupling and replay capability. 3. Implement an API versioning strategy (URI versioning: /api/v1/jobs, /api/v2/jobs) from day one. 4. Add a service mesh (Istio) for advanced traffic management and mTLS between services. 5. Implement CQRS — separate read (query) models from write (command) models for better scalability.

■ SECTION 6: DEVOPS + CLOUD + TOOLS

Q1.

Explain the Dockerfile you wrote for your Spring Boot service.

```
# Multi-stage build for smaller image FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app COPY pom.xml . RUN mvn dependency:go-offline # Cache dependencies
layer COPY src ./src RUN mvn package -DskipTests FROM
eclipse-temurin:17-jre-alpine # Smaller runtime image WORKDIR /app COPY
--from=builder /app/target/*.jar app.jar EXPOSE 8080 ENTRYPOINT ["java", "-jar",
"app.jar"]
```

◆ **Model Answer**

Multi-stage builds reduce final image size (only JRE, not full JDK). Caching the dependency layer (COPY pom.xml + RUN mvn dependency:go-offline) means rebuilds are fast — only re-downloads if pom.xml changes.

Q2.

What is a Kubernetes Pod, Deployment, and Service?

◆ **Model Answer**

Pod: Smallest deployable unit — one or more containers sharing network and storage. Deployment: Manages a ReplicaSet — ensures N replicas of your Pod are always running. Handles rolling updates and rollbacks. Service: Provides a stable network endpoint to a set of Pods. Types: ClusterIP (internal), NodePort (external via node port), LoadBalancer (external via cloud LB), ExternalName (DNS alias).

Q3.

What is a CI/CD pipeline? Describe the one you used.

◆ **Model Answer**

CI (Continuous Integration): Every code push triggers: code compilation, unit tests, SonarQube static analysis, Docker image build. CD (Continuous Delivery/Deployment): On merge to main: integration tests, Docker image push to registry, Kubernetes deployment update. My pipeline (GitHub Actions): Push → Build → Test → Docker Build → Push to Docker Hub → kubectl apply deployment. Each step must pass for the next to run. Failed tests block the deploy.

```
# .github/workflows/deploy.yml (simplified) on: push: branches: [main] jobs:
build-test-deploy: runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 -
name: Build & Test run: mvn clean verify - name: Docker Build & Push run: | docker
build -t viraj/job-service:${{ github.sha }} . docker push viraj/job-service:${{
github.sha }} - name: Deploy to K8s run: kubectl set image deployment/job-service
job-service=viraj/job-service:${{ github.sha }}
```

Q4.

What is Kubernetes HPA?

◆ **Model Answer**

HPA (Horizontal Pod Autoscaler) automatically scales the number of Pod replicas based on metrics (CPU, memory, or custom). Example: If CPU > 70% for 2 minutes, add 2 more replicas (max 10). If CPU drops below 30%, scale down to minimum (2). Configuration is done via kubectl autoscale or a HorizontalPodAutoscaler YAML manifest.

Q5.

What are Kubernetes ConfigMaps and Secrets?

◆ Model Answer

ConfigMap: Stores non-sensitive configuration as key-value pairs — injected as environment variables or mounted as files. Example: database URL, service endpoints. Secret: Stores sensitive data (base64 encoded, not encrypted by default — use etcd encryption). Example: DB password, JWT secret. In Spring Boot, reference them via `${DB_PASSWORD}` in `application.properties` — the value comes from the K8s environment.

Q6.

How do you manage logs in a microservices environment?

◆ Model Answer

Each service logs to stdout/stderr. Kubernetes collects these via Fluentd/Fluentbit and ships to Elasticsearch. Kibana provides a UI to search and visualize logs. Critical practice: Use a structured logging format (JSON) so Kibana can parse fields. Add a correlation/trace ID to every log line so you can trace a request across services: `log.info('Applying for job | traceId={} userId={} jobId={}', traceId, userId, jobId)`.

Q7.

What is Maven? How does the build lifecycle work?

◆ Model Answer

Maven manages Java project builds and dependencies. Key lifecycle phases: `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`. `mvn clean package`: Compiles code, runs tests, creates JAR/WAR in `target/`. `mvn clean verify`: Also runs integration tests. `mvn install`: Copies artifact to local .m2 repository. Dependencies declared in `pom.xml` are downloaded from Maven Central. I used `dependencyManagement` section to enforce consistent versions across modules.

■ SECTION 7: CODING ROUND

Solve in 20–25 minutes total. Think aloud. Discuss brute force, then optimize.

Problem 1 — Two Sum (Easy)

Given an array of integers and a target, return the indices of two numbers that add up to the target. Assume exactly one solution exists.

Approach: Use a HashMap to store each element and its index. For each element, check if (target - element) is already in the map. O(n) time, O(n) space.

```
public int[] twoSum(int[] nums, int target) { Map<Integer, Integer> map = new
HashMap<>(); for (int i = 0; i < nums.length; i++) { int complement = target -
nums[i]; if (map.containsKey(complement)) { return new int[]{map.get(complement),
i}; } map.put(nums[i], i); } throw new IllegalArgumentException("No solution"); }
```

Follow-up: What if the array was sorted? (Use two pointers — O(1) space)

Problem 2 — Reverse Words in a String (Easy-Medium)

Given 's = the sky is blue', return 'blue is sky the'. Trim extra spaces.

```
public String reverseWords(String s) { String[] words = s.trim().split("\\s+"); //
handles multiple spaces StringBuilder sb = new StringBuilder(); for (int i =
words.length - 1; i >= 0; i--) { sb.append(words[i]); if (i > 0) sb.append(" "); }
return sb.toString(); }
```

Follow-up: Do it in-place without split()? (Character array + two-pointer reverse)

Problem 3 — Group Anagrams (Medium)

Given ['eat','tea','tan','ate','nat','bat'], group anagrams together.

Approach: Sort each word → use sorted word as key in a HashMap. Anagrams have the same sorted form. O(n * k log k) where k is max word length.

```
public List<List<String>> groupAnagrams(String[] strs) { Map<String, List<String>>
map = new HashMap<>(); for (String s : strs) { char[] chars = s.toCharArray();
Arrays.sort(chars); String key = new String(chars); // sorted form as key
map.computeIfAbsent(key, k -> new ArrayList<>()).add(s); } return new
ArrayList<>(map.values()); }
```

Follow-up: Can you do it without sorting? (Use character frequency array as key)

Problem 4 — Find Duplicate in Array (Medium)

Given array of n+1 integers, each between 1 and n, find the duplicate without modifying the array and using O(1) extra space.

Approach: Floyd's cycle detection (fast & slow pointer — treat array as a linked list).

```
public int findDuplicate(int[] nums) { int slow = nums[0], fast = nums[0]; //
Phase 1: Find intersection do { slow = nums[slow]; fast = nums[nums[fast]]; }
while (slow != fast); // Phase 2: Find entrance to cycle slow = nums[0]; while
(slow != fast) { slow = nums[slow]; fast = nums[fast]; } return slow; }
```

Follow-up: What if there could be multiple duplicates? (Use a HashSet — O(n) space)

■ SECTION 8: RAPID FIRE + CROSS QUESTIONS

Answer in 30–60 seconds each. Be precise.

Q1	What is the default scope of a Spring bean?	Singleton — one instance per Spring container.
Q2	What is @Autowired?	Injects a Spring-managed bean. Spring resolves by type, then by name.
Q3	Difference: PUT vs PATCH?	PUT replaces the entire resource. PATCH applies partial updates.
Q4	What is idempotency?	Calling same request N times has same effect as calling it once. GET, PUT, DELETE are idempotent. POST is not.
Q5	What is a DTO?	Data Transfer Object — a POJO used to transfer data between layers, hiding entity internals from the API.
Q6	What does @Transient mean in JPA?	Marks a field to be excluded from DB persistence.
Q7	What is HikariCP?	High-performance JDBC connection pool. Default in Spring Boot. Manages DB connection reuse.
Q8	HTTP 200 vs 201 vs 204?	200: OK. 201: Created (after POST). 204: No Content (after DELETE).
Q9	What is a K8s Namespace?	Logical isolation within a cluster — separate dev, staging, prod environments.
Q10	What is Spring Cloud Config?	Centralized external config server. Services fetch config at startup from a Git repo.
Q11	Difference: @PathVariable vs @RequestParam?	PathVariable extracts from URI path. RequestParam extracts from query string.
Q12	What is orphanRemoval in JPA?	When a child entity is removed from the parent's collection, it's automatically deleted from DB.

■ SECTION 9: INTERVIEWER EVALUATION GUIDE

What Interviewers Look For

- Structured Communication: Can you explain a concept step-by-step without rambling?
- Depth over Breadth: Do you know WHY something works, not just WHAT it is?
- Project Ownership: Can you speak confidently about technical decisions you made?
- Problem-Solving Mindset: Do you break problems into parts before coding?
- Quantified Impact: Do you use numbers to describe your contributions?
- Intellectual Curiosity: Do you mention what you'd do differently or what you're learning?

Common Mistakes to Avoid

- Saying 'I used X' without being able to explain HOW X works internally.
- Giving only theory without connecting it to your project experience.
- Not asking clarifying questions before coding.
- Ignoring edge cases in code solutions (null input, empty array, negative numbers).
- Being passive in behavioral answers — use 'I' not 'we' for your specific contribution.
- Freezing on unknown questions — say 'I haven't used X in production but I understand it as...'

How to Stand Out

- Reference your Job Portal project for every relevant technical question.
- Use technical vocabulary naturally: 'We used cursor-based pagination because offset is $O(n)$ at deep pages.'
- Show trade-off awareness: 'We chose X over Y because of Z, though Y would be better if...'
- Ask intelligent questions at the end: 'What does the tech stack look like at the team I'd join?'
- Mention the non-obvious: monitoring, tracing, graceful degradation — shows production experience.

All the best, Viraj! You've got the experience and the project — now own the narrative. ■